

Des règles pour ton agent IA, et un verrou qui les fait respecter. En 30 minutes.

Microsoft, Work Trend Index du 5 mai 2026 : « People are ready. The systems around them are not. » Chacun a son agent, et rien d'écrit sur ce qu'il sait ni sur ce qu'il peut faire.

- Quatre notes avant tout agent : qui on est, nos objectifs, notre environnement, nos règles.
- Les règles tiennent en quatre lignes. Une règle écrite seule reste un conseil.
- Un hook la rend exécutable : il refuse la commande interdite avant qu'elle parte.
- À la fin : tes quatre notes, ton hook branché, et la preuve qu'il refuse.

Aucune ligne de commande. Chaque bloc gris se copie tel quel dans la fenêtre de Claude.

Une règle lue il y a une heure ne pèse plus rien.

Chez nous, « aucun message à un tiers sans relecture » existait depuis le premier jour. Au bout d'une longue session, l'agent a quand même proposé d'envoyer. La règle s'était diluée dans la conversation.

Où vit la règle	Ce qu'elle vaut
Dans un fichier de consignes	un conseil, qui s'oublie en route
Dans un hook	un refus, même si la consigne est oubliée
Dans un hook testé	un refus prouvé, qui tient après tes changements

Nos quatre règles

Jamais un secret dans une note. Toute campagne créée en pause. Aucune commande git destructive. Une validation humaine avant toute suppression de masse.

Un agent qui tourne la nuit prépare. Le matin, on décide.

Installe Claude et ouvre ton dossier.

1

Installe l'application

Va sur claude.ai/download, installe l'application pour Mac ou Windows, ouvre-la et connecte-toi.

2

Crée le dossier

Dans Documents, clic droit, « Nouveau dossier », appelle-le regles.

3

Ouvre-le dans l'onglet Code

En haut de l'application, clique sur Code, choisis « Local », puis sélectionne le dossier regles.

4

Choisis le mode prudent

Prends le mode « Manual » : l'agent te demande avant chaque modification.

Tu dois voir : une zone de message, avec le nom du dossier regles affiché.

Fais écrire les quatre notes et les règles.

Copie ce message et envoie-le. L'agent crée les notes, puis te pose les questions une par une. Réponds en quelques mots : c'est lui qui met en forme.

Message à envoyer

Crée quatre notes d'une page maximum :
00–qui–on–est.md, 01–objectifs.md, 02–environnement.md, 03–regles.md.
Remplis les trois premières en me posant les questions une par une.
N'invente rien : si je ne sais pas, écris « à compléter ».

Dans 03–regles.md, écris ces quatre règles :

1. Jamais un secret ni un mot de passe dans une note.
2. Toute campagne publicitaire est créée en pause.
3. Aucune commande git destructive.
4. Validation humaine avant toute suppression de masse.

Puis crée CLAUDE.md : « Au début de chaque conversation,
lis les quatre notes. »

Tu dois voir : quatre notes et un fichier CLAUDE.md. Une règle que tu répètes souvent à ton agent ?
Ajoute-la en cinquième ligne, pas plus.

Fais construire le hook par l'agent.

Le hook lit chaque commande avant qu'elle parte. Tu n'as rien à écrire toi-même : copie ce message, et accepte chaque fichier quand l'agent te le demande.

Message à envoyer

```
Crée un hook de type PreToolUse pour ce dossier,
déclaré dans .claude/settings.json.
Il lit chaque commande avant qu'elle s'exécute et refuse :
les suppressions en masse, git push --force,
git reset --hard, et tout envoi de mail.
Quand il refuse, il dit pourquoi : « refusé, demande un accord ».
Tout le reste passe.
Écris-le dans un langage qui marche sur mon ordinateur,
rends-le exécutable, et explique-moi en trois phrases ce qu'il fait.
```

Tu dois voir : deux fichiers dans le dossier .claude, et l'explication. Le dossier .claude est masqué : sur Mac, Cmd + Maj + point l'affiche.

La preuve : le hook refuse-t-il vraiment ?

Un hook présent n'est pas un hook qui bloque. Ouvre une nouvelle conversation dans le même dossier, puis copie ce message.

Message à envoyer

Écris un test qui rejoue trois gestes contre le hook :
git push --force origin main (doit être refusé),
une suppression en masse d'un dossier d'essai (refusée),
git status (doit passer).
Lance le test et donne-moi le résultat ligne par ligne.

Ce qu'il répond

git push --force origin main : refusé, demande un accord
suppression en masse : refusé, demande un accord
git status : autorisé
Hook vérifié : 2 refus, 1 autorisation.

Si...	Fais ça
Un geste interdit passe	Nouvelle conversation, et redemande-lui le test
Tout est bloqué	Demande-lui de corriger : git status doit passer
Tu ajoutes une règle	Ajoute aussi sa ligne dans le test

Tu dois voir : deux refus et une autorisation. C'est ton résultat : tes règles ne dépendent plus de la mémoire de l'agent.

Trois phrases, et la suite.

- Une règle dans un document est une intention, pas une garantie.
- Ce qui est irréversible se bloque avant l'exécution, pas après.
- Un hook sans test devient décoratif sans que personne le remarque.

Et après ?

Relance le test à chaque changement du hook. Quand tu branches un outil qui envoie ou paie, ajoute son geste au hook et une ligne au test.

Le piège

Écrire vingt règles d'un coup. Quatre règles branchées sur un hook protègent mieux que vingt que personne n'applique.

Quelle règle répètes-tu le plus souvent à ton agent ? Réponds à mon message, on regarde si elle peut devenir un refus.